

Pandas in Python

- Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. In 2008, developer Wes McKinney started developing pandas when in need of high performance, flexible tool for analysis of data. This library is built on the top of the NumPy library. Pandas generally provide two data structure for manipulating data, They are:
 - Series
 - DataFrame

Series

- Series is a one-dimensional labeled array capable of holding any data type (integers, strings, floating point numbers, Python objects, etc.).
- The basic method to create a Series is to call:

```
s = pd.Series(data, index=index) Here, data can be many different and index is an optional argument
```

- a Python dict
- an ndarray
- a scalar value (like 5)
- list Index values must be unique and hashable, same length as data. Default np.arange(n) if no index is passed.

```
In [2]: import numpy as np
import pandas as pd
```

Create an Empty Series

```
In [3]: s = pd.Series()
print(s)
```

C:\Users\gcet\AppData\Local\Temp\ipykernel_1040\926796085.py:1: FutureWarning: The default dtype for empty Series will be 'object' instead of 'float64' in a future version. Specify a dtype explicitly to silence this warning.

```
s = pd.Series()

Series([], dtype: float64)
```

Create a Series from ndarray

If data is an ndarray, then index passed must be of the same length. If no index is passed, then by default index will be range(n) where n is array length, i.e., [0,1,2,3.... range(len(array))-1].

```
In [4]: data = np.array(['a', 'b', 'c', 'd'])
s = pd.Series(data)
print(s)
# We did not pass any index, so by default, it assigned the indexes ranging from 0 to len(data)-1, i.e., 0 to 3.
```

```
0    a
1    b
2    c
3    d
dtype: object
```

```
In [5]: data = np.array(['a', 'b', 'c', 'd'])
s = pd.Series(data, index=[100, 101, 102, 103])
print(s)
# We passed the index values here. Now we can see the customized indexed values in the output.
```

```
100    a
101    b
102    c
103    d
dtype: object
```

Create a Series from dict

A dict can be passed as input and if no index is specified, then the dictionary keys are taken in a sorted order to construct index. If index is passed, the values in data corresponding to the labels in the index will be pulled out.

```
In [6]: data = {'a' : 0., 'b' : 1., 'c' : 2.}
s = pd.Series(data)
print(s)
```

```
a    0.0
b    1.0
c    2.0
dtype: float64
```

```
In [7]: data = {'a' : 0., 'b' : 1., 'c' : 2.}
s = pd.Series(data,index=['b','c','d','a'])
print(s)

b    1.0
c    2.0
d     NaN
a     0.0
dtype: float64
```

Create a Series from Scalar

If data is a scalar value, an index must be provided. The value will be repeated to match the length of index

```
In [8]: s = pd.Series(5, index=[0, 1, 2, 3])
print(s)

0    5
1    5
2    5
3    5
dtype: int64
```

Creating series from list of values

```
In [9]: data = pd.Series([0.25, 0.5, 0.75, 1.0])
data
```

```
Out[9]: 0    0.25
1    0.50
2    0.75
3    1.00
dtype: float64
```

```
In [10]: data.values
```

```
Out[10]: array([0.25, 0.5 , 0.75, 1.  ])
```

```
In [11]: data.index
```

```
Out[11]: RangeIndex(start=0, stop=4, step=1)
```

Accessing Data from Series with Position

Data in the series can be accessed similar to that in an ndarray. Retrieve the first element. As we already know, the counting starts from zero for the array, which means the first element is stored at zeroth position and so on.

```
In [12]: s = pd.Series([1,2,3,4,5],index = ['a','b','c','d','e'])

#retrieve the first element
print(s[0])
#retrieve the first three element
print(s[:3])
#retrieve the last three element
print(s[-3:])

1
a    1
b    2
c    3
dtype: int64
c    3
d    4
e    5
dtype: int64
```

Retrieve Data Using Label (Index)

```
In [13]: s = pd.Series([1,2,3,4,5],index = ['a','b','c','d','e'])

#retrieve a single element
print(s['a'])
#retrieve multiple elements
print(s[['a','c','d']])
#print s['f'] raises error as KeyError: 'f'

1
a    1
c    3
d    4
dtype: int64
```

Dataframe in Pandas

- A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns.
- Features of DataFrame:

- Potentially columns are of different types
- Size – Mutable
- Labeled axes (rows and columns)
- Can Perform Arithmetic operations on rows and columns

In the real world, a Pandas DataFrame will be created by loading the datasets from existing storage, storage can be SQL Database, CSV file, and Excel file. Pandas DataFrame can be created from the lists, dictionary, and from a list of dictionary etc.

Create DataFrame

A pandas DataFrame can be created using various inputs like –

- Lists
- dict
- Series
- Numpy ndarrays
- Another DataFrame

Create an Empty DataFrame

```
In [14]: import pandas as pd
df = pd.DataFrame()
print(df)
```

```
Empty DataFrame
Columns: []
Index: []
```

create a Pandas DataFrame

```
In [15]: data = [1,2,3,4,5]
df = pd.DataFrame(data)
print(df)
df = pd.DataFrame(data,index=['one','two','three','four','five'])
print(df)
```

```
0
0 1
1 2
2 3
3 4
4 5

0
one 1
two 2
three 3
four 4
five 5
```

```
In [16]: data = [['Alex',10],['Bob',12],['Clarke',13]]
df = pd.DataFrame(data,columns=['Name','Age'],dtype=float)
print(df)
print(df.columns)
```

```
   Name  Age
0  Alex  10.0
1   Bob  12.0
2 Clarke 13.0
Index(['Name', 'Age'], dtype='object')
```

C:\Users\gcet\AppData\Local\Temp\ipykernel_1040\2619782359.py:2: FutureWarning: Could not cast to float64, falling back to object. This behavior is deprecated. In a future version, when a dtype is passed to 'DataFrame', either all columns will be cast to the dtype, or a TypeError will be raised.

```
df = pd.DataFrame(data,columns=['Name','Age'],dtype=float)
```

```
In [17]: #creating dataframe from dictionary
my_dict = {
    'name' : ["a", "b", "c", "d", "e", "f", "g"],
    'age' : [20,27, 35, 55, 18, 21, 35],
    'designation': ["VP", "CEO", "CEO", "VP", "VP", "CEO", "MD"]
}
df = pd.DataFrame(my_dict)
print(df)
```

```
   name  age designation
0    a   20          VP
1    b   27          CEO
2    c   35          CEO
3    d   55          VP
4    e   18          VP
5    f   21          CEO
6    g   35          MD
```

```
In [18]: #Create a DataFrame from List of Dicts
data = [{ 'a': 1, 'b': 2},{ 'a': 5, 'b': 10, 'c': 20}]
df = pd.DataFrame(data)
print(df) #Note - Observe, NaN (Not a Number) is appended in missing areas.
```

	a	b	c
0	1	2	NaN
1	5	10	20.0

```
In [19]: #Create a DataFrame from Dict of Series
d = { 'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),
      'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])}

df = pd.DataFrame(d)
print(df)
```

	one	two
a	1.0	1
b	2.0	2
c	3.0	3
d	NaN	4

The Columns of Pandas DataFrame

Unlike python lists or dictionaries and just like NumPy, a column of the DataFrame will always be of same type.

```
In [20]: print(df['one'].dtype)      # Dict Like Syntax
print(df.one.dtype)              # DataFrame.ColumnName
print(df.two.dtype)
print(df.dtypes) #check the data types of all columns
```

```
float64
float64
int64
one    float64
two    int64
dtype: object
```

Viewing the Data of a DataFrame

- Pandas DataFrame will contain hundreds (if not thousands) of rows of data. We can only view them selectively at any point of time. To selectively view the rows, we can use head(...) and tail(...) functions, which by default give first or last five rows (if no input is provided), otherwise shows specific number of rows from top or bottom

```
In [91]: my_dict = {
          'name' : ["a", "b", "c", "d", "e", "f", "g"],
          'age'  : [20, 27, 35, 55, 18, 21, 35],
          'designation': ["VP", "CEO", "CEO", "VP", "VP", "CEO", "MD"]
        }
df = pd.DataFrame(my_dict, index=[0,1,2,3,4,5,6])
print(df)
```

	name	age	designation
0	a	20	VP
1	b	27	CEO
2	c	35	CEO
3	d	55	VP
4	e	18	VP
5	f	21	CEO
6	g	35	MD

The head() function

It is primarily used to view the first few rows of a dataset. It helps users quickly get an overview of the data and its structure.

```
In [92]: df.head() # Displays 1st Five Rows
```

Out[92]:

	name	age	designation
0	a	20	VP
1	b	27	CEO
2	c	35	CEO
3	d	55	VP
4	e	18	VP

The tail() function offers a rapid view of the final few rows of a dataset

```
In [93]: df.tail()    ## Displays Last Five Rows
```

```
Out[93]:
```

	name	age	designation
2	c	35	CEO
3	d	55	VP
4	e	18	VP
5	f	21	CEO
6	g	35	MD

```
In [94]: df.head(2)   # Displays 1st two Rows
```

```
Out[94]:
```

	name	age	designation
0	a	20	VP
1	b	27	CEO

```
In [95]: df.tail(3)   # Displays Last 3 Rows
```

```
Out[95]:
```

	name	age	designation
4	e	18	VP
5	f	21	CEO
6	g	35	MD

shape property

- Return a tuple representing the dimensionality of the DataFrame.

```
In [96]: df.shape
```

```
Out[96]: (7, 3)
```

pandas.DataFrame.info()

This method prints information about a DataFrame including the index dtype and columns, non-null values and memory usage.

```
In [97]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column        Non-Null Count  Dtype
---  -
 0   name          7 non-null     object
 1   age           7 non-null     int64
 2   designation    7 non-null     object
dtypes: int64(1), object(2)
memory usage: 224.0+ bytes
```

```
In [98]: print(df.index)    # For Row Indexes
```

```
Int64Index([0, 1, 2, 3, 4, 5, 6], dtype='int64')
```

Pandas DataFrame.columns

This attribute does not require any parameters to be passed. When called on a data frame using the syntax DataFrame.columns, it returns the names of the columns present in that data frame.

```
In [99]: print(df.columns)    # For Columns names
```

```
Index(['name', 'age', 'designation'], dtype='object')
```

Statistics

- df.describe() | Summary statistics for numerical columns
- df.mean() | Returns the mean of all columns
- df.corr() | Returns the correlation between columns in a DataFrame
- df.count() | Returns the number of non-null values in each DataFrame column
- df.max() | Returns the highest value in each column
- df.min() | Returns the lowest value in each column
- df.median() | Returns the median of each column
- df.std() | Returns the standard deviation of each column

In [100]: `df.describe()`

Out[100]:

	age
count	7.000000
mean	30.142857
std	12.966991
min	18.000000
25%	20.500000
50%	27.000000
75%	35.000000
max	55.000000

In [101]: `df['age'].mean()`

Out[101]: 30.142857142857142

Selecting data-- pandas data access methods, .at, .iat, .loc and .iloc

In [102]: *#Column Selection*
`df['name']`

Out[102]:

0	a
1	b
2	c
3	d
4	e
5	f
6	g

Name: name, dtype: object

In [103]: `df[['name', 'age']]`

Out[103]:

	name	age
0	a	20
1	b	27
2	c	35
3	d	55
4	e	18
5	f	21
6	g	35

In [104]: *# to get column data in form of array*
`df['name'].values`

Out[104]: array(['a', 'b', 'c', 'd', 'e', 'f', 'g'], dtype=object)

In [105]: *#Selecting rows in a DataFrame*
`df[0:3]`

Out[105]:

	name	age	designation
0	a	20	VP
1	b	27	CEO
2	c	35	CEO

Selecting data using “iloc”

- The iloc indexer for Pandas Dataframe is used for integer-location based indexing / selection by position.
- The iloc indexer syntax is `data.iloc[ , ]`.

In [106]: *#Single selections using iloc and DataFrame*
#row selection
`df.iloc[0]`

Out[106]:

name	a
age	20
designation	VP

Name: 0, dtype: object

In [107]: `df.iloc[:, :-1]`

Out[107]:

	designation	age	name
0	VP	20	a
1	CEO	27	b
2	CEO	35	c
3	VP	55	d
4	VP	18	e
5	CEO	21	f
6	MD	35	g

In [108]: `#row selection`
`df.iloc[-1]`

Out[108]:

name	g
age	35
designation	MD

Name: 6, dtype: object

In [109]: `#column selection`
`df.iloc[:, 0]`

Out[109]:

0	a
1	b
2	c
3	d
4	e
5	f
6	g

Name: name, dtype: object

In [110]: `#multiple rows and columns`
`df.iloc[0:3, -1]`

Out[110]:

0	VP
1	CEO
2	CEO

Name: designation, dtype: object

In [111]: `df.iloc[[1,3],[0,1]]`

Out[111]:

	name	age
1	b	27
3	d	55

Selecting data using “loc”

- loc is primarily label based indexing. Integers may be used but they are interpreted as a label.

In [112]: `df.loc[[0,5], :]`

Out[112]:

	name	age	designation
0	a	20	VP
5	f	21	CEO

Conditional selections of data using "loc "

In [113]: `df.loc[df['name']=='a', ['age', 'designation']]`

Out[113]:

	age	designation
0	20	VP

In [114]: `df`

Out[114]:

	name	age	designation
0	a	20	VP
1	b	27	CEO
2	c	35	CEO
3	d	55	VP
4	e	18	VP
5	f	21	CEO
6	g	35	MD

```
In [115]: df.loc[(df['age']>30) & (df['age']<40),['designation']]
```

```
Out[115]:
```

	designation
2	CEO
6	MD

```
In [116]: print(df[df['age'].between(15, 20)])
```

	name	age	designation
0	a	20	VP
4	e	18	VP

The Pandas DataFrame.set_index()

This method is used to assign a list, series, or another data frame as the index of a given data frame. It is particularly useful when combining multiple data frames, allowing for easy modification of the index. While an index column can be specified during data frame creation, set_index() provides a flexible way to change the index later on.

```
In [117]: df.set_index("name",inplace=True)
print(df)
```

	age	designation
name		
a	20	VP
b	27	CEO
c	35	CEO
d	55	VP
e	18	VP
f	21	CEO
g	35	MD

```
In [118]: df.loc['a']
```

```
Out[118]: age      20
designation  VP
Name: a, dtype: object
```

The reset_index method

It allows you to reset the index to the default integer-based index and reset the index in Pandas DataFrame

```
In [119]: df.reset_index(inplace=True)
print(df)
```

	name	age	designation
0	a	20	VP
1	b	27	CEO
2	c	35	CEO
3	d	55	VP
4	e	18	VP
5	f	21	CEO
6	g	35	MD

Multi-Index DataFrame

Hierarchical indexes (AKA multiindexes) help us to organize, find, and aggregate information faster at almost no cost. Example where we need Hierarchical indexes

```
In [120]: df.set_index(["name", "age"],inplace=True)
print(df)
```

		designation
name	age	
a	20	VP
b	27	CEO
c	35	CEO
d	55	VP
e	18	VP
f	21	CEO
g	35	MD

Data retrieval based on the multi indexing

```
In [121]: print(df.loc['a',20])
```

```
designation    VP
Name: (a, 20), dtype: object
```

Selecting pandas data using ix

- The ix[] indexer is a hybrid of .loc and .iloc. Generally, ix is label based and acts just as the .loc indexer. However, .ix also supports integer type selections (as in .iloc) where passed an integer. This only works where the index of the DataFrame is not integer based. ix will accept any of the inputs of .loc and .iloc.

- The ix indexer has been **deprecated** in recent versions of Pandas,

Column Addition--adding a new column to an existing data frame

```
In [122]: df.reset_index(inplace=True)
print(df)
```

	name	age	designation
0	a	20	VP
1	b	27	CEO
2	c	35	CEO
3	d	55	VP
4	e	18	VP
5	f	21	CEO
6	g	35	MD

```
In [123]: df['experience']=[3,4,4,5,6,6,7]
print(df)
print(df.columns)
```

	name	age	designation	experience
0	a	20	VP	3
1	b	27	CEO	4
2	c	35	CEO	4
3	d	55	VP	5
4	e	18	VP	6
5	f	21	CEO	6
6	g	35	MD	7

Index(['name', 'age', 'designation', 'experience'], dtype='object')

Column Deletion--Columns can be deleted or popped

```
In [124]: del df['experience']
print(df)
```

	name	age	designation
0	a	20	VP
1	b	27	CEO
2	c	35	CEO
3	d	55	VP
4	e	18	VP
5	f	21	CEO
6	g	35	MD

Sorting data

```
In [125]: df.sort_values('age')
```

```
Out[125]:
```

	name	age	designation
4	e	18	VP
0	a	20	VP
5	f	21	CEO
1	b	27	CEO
2	c	35	CEO
6	g	35	MD
3	d	55	VP

```
In [126]: df.sort_values(by=['designation', 'age'], ascending=[False, True])
```

```
Out[126]:
```

	name	age	designation
4	e	18	VP
0	a	20	VP
3	d	55	VP
6	g	35	MD
5	f	21	CEO
1	b	27	CEO
2	c	35	CEO

```
In [127]: df['name'] = df['name'].replace('b', 'b1')
print(df)
```

	name	age	designation
0	a	20	VP
1	b1	27	CEO
2	c	35	CEO
3	d	55	VP
4	e	18	VP
5	f	21	CEO
6	g	35	MD

```
In [128]: df['experience']=[3,4,4,5,6,6,7]
print(df)
print(df.columns)
```

```
   name  age  designation  experience
0    a   20           VP           3
1   b1   27           CEO           4
2    c   35           CEO           4
3    d   55           VP           5
4    e   18           VP           6
5    f   21           CEO           6
6    g   35           MD           7
Index(['name', 'age', 'designation', 'experience'], dtype='object')
```

pandas.DataFrame.groupby

A groupby operation involves some combination of splitting the object, applying a function, and combining the results. This can be used to group large amounts of data and compute operations on these groups. The 3 Steps of a Groupby Process

Any groupby process involves some combination of the following 3 steps:

- **Splitting** the original object into groups based on the defined criteria.
- **Applying** a function to each group.
- **Combining** the results.

Applying a Function by Group

After splitting the original data and (optionally) inspecting the resulting groups, we can perform one of the following operations or their combination (not necessarily in the given order) on each group:

- **Aggregation:** calculating a summary statistic for each group (e.g., group sizes, means, medians, or sums) and outputting a single number for many data points.
- **Transformation:** conducting some operations by group, such as calculating the z-score for each group.
- **Filtration:** rejecting some groups based on a predefined condition, such as group size, mean, median, or sum. This can also include filtering out particular rows from each group.

Splitting the Original Object into Groups

At this stage, we call the pandas DataFrame.groupby() function. We use it to split the data into groups based on predefined criteria, along rows (by default, axis=0), or columns (axis=1). In other words, this function maps the labels to the names of the groups.

```
In [129]: grouped=df.groupby('designation')
print(grouped)
list(grouped)
```

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x00000E11120A310>
```

```
Out[129]: [(<'CEO',
      name age designation  experience
1    b1   27           CEO           4
2     c   35           CEO           4
5     f   21           CEO           6),
 (<'MD',
      name age designation  experience
6     g   35           MD           7),
 (<'VP',
      name age designation  experience
0     a   20           VP           3
3     d   55           VP           5
4     e   18           VP           6)]
```

To briefly inspect the resulting GroupBy object and check how exactly the groups were split, we can extract from it the groups or indices attribute. Both of them return a dictionary where the keys are the created groups and the values are the lists of axis labels (for the groups attribute) or indices (for the indices attribute) of the instances of each group in the original DataFrame:

```
In [130]: grouped.indices
```

```
Out[130]: {'CEO': array([1, 2, 5], dtype=int64),
          'MD': array([6], dtype=int64),
          'VP': array([0, 3, 4], dtype=int64)}
```

To find the number of groups in a GroupBy object, we can extract from it the ngroups attribute or call the len function of the Python standard library:

```
In [131]: grouped.ngroups
len(grouped)
```

```
Out[131]: 3
```

In [132]:

grouped.get_group("CEO")

Out[132]:

	name	age	designation	experiance
1	b1	27	CEO	4
2	c	35	CEO	4
5	f	21	CEO	6

Aggregation

To aggregate the data of a GroupBy object (i.e., to calculate a summary statistic by group), we can use the agg() method on the object:

In [133]:

grouped.agg(np.mean)

C:\Users\gcet\AppData\Local\Temp\ipykernel_1040\1243003623.py:1: FutureWarning: The operation <function mean at 0x000000E1071C2520> failed on a column. If any error is raised, this will raise an exception in a future version of pandas. Drop these columns to avoid this warning.
grouped.agg(np.mean)

Out[133]:

	age	experiance
designation		
CEO	27.666667	4.666667
MD	35.000000	7.000000
VP	31.000000	4.666667

The code above produces a DataFrame with the group names as its new index and the mean values for each numeric column by group.

Instead of using the agg() method, we can apply the corresponding pandas method directly on a GroupBy object. The most common methods are mean(), median(), mode(), sum(), size(), count(), min(), max(), std(), var() (computes the variance of each group), describe() (outputs descriptive statistics by group), and nunique() (gives the number of unique values in each group).

In [134]:

grouped.sum()

C:\Users\gcet\AppData\Local\Temp\ipykernel_1040\2368404282.py:1: FutureWarning: The default value of numeric_only in DataFrameGroupBy.sum is deprecated. In a future version, numeric_only will default to False. Either specify numeric_only or select only columns which should be valid for the function.
grouped.sum()

Out[134]:

	age	experiance
designation		
CEO	83	14
MD	35	7
VP	93	14

Usually, we are interested only in the statistics for some particular column or columns, so we need to specify it (or them).

In [135]:

grouped['age'].sum()

Out[135]:

designation	age
CEO	83
MD	35
VP	93
Name: age, dtype: int64	

In [136]:

grouped[['age', 'experiance']].sum()

Out[136]:

	age	experiance
designation		
CEO	83	14
MD	35	7
VP	93	14

It is possible to apply several functions at once to one or more columns of a GroupBy object. For this purpose, we again need the agg() method and a list of the functions of interest:

In [137]:

grouped[['age', 'experiance']].agg([np.sum, np.mean, np.std])

Out[137]:

	age			experiance		
	sum	mean	std	sum	mean	std
designation						
CEO	83	27.666667	7.023769	14	4.666667	1.154701
MD	35	35.000000	NaN	7	7.000000	NaN
VP	93	31.000000	20.808652	14	4.666667	1.527525

Also, we can consider applying different aggregation functions to different columns of a GroupBy object by passing a dictionary:

```
In [138]: grouped.agg({'age': [np.sum, np.size], 'experience': np.mean})
```

```
Out[138]:
```

		age		experience
		sum	size	mean
designation				
	CEO	83	3	4.666667
	MD	35	1	7.000000
	VP	93	3	4.666667

Transformation

Unlike aggregation methods (and also unlike filtration ones, as we will see soon), transformation methods return a new DataFrame with the same shape and indexing as the original one but with transformed individual values. Here it is important to note that the transformation must not modify any values in the original DataFrame, meaning that such operations cannot be performed in place.

The most common pandas method to transform the data of a GroupBy object is transform(). For example, it can be helpful for computing the z-score for each group:

```
In [139]: grouped[['age', 'experience']].transform(lambda x: (x - x.mean()) / x.std())
```

```
Out[139]:
```

	age	experience
0	-0.528626	-1.091089
1	-0.094916	-0.577350
2	1.044074	-0.577350
3	1.153366	0.218218
4	-0.624740	0.872872
5	-0.949158	1.154701
6	NaN	NaN

With transformation methods, we can also replace missing data with the group mean, median, mode, or any other value:

```
In [146]: grouped[['age', 'experience']].transform(lambda x: x.fillna(x.mode()[0]))
```

```
Out[146]:
```

	age	experience
0	20	3
1	27	4
2	35	4
3	55	5
4	18	6
5	21	6
6	35	7

There are some other pandas methods that we can use to transform the data of a GroupBy object: bfill(), ffill(), diff(), pct_change(), rank(), shift(), quantile(), etc.

Filtration

Filtration methods discard the groups or particular rows from each group based on a predefined condition and return a subset of the original data. For example, we may want to keep only the values of a certain column from all the groups where the group mean for that column is greater than a predefined value. In the case of our DataFrame, let's filter out all the groups with a group mean for the prizeAmountAdjusted column smaller than 7,000,000, and keep only this column in the output:

```
In [158]: grouped['age'].filter(lambda x: x.mean() > 21)
```

```
Out[158]:
```

0	20
1	27
2	35
3	55
4	18
5	21
6	35

Name: age, dtype: int64

Renaming Columns in DataFrame

```
In [88]: df.rename(columns={"age": "Age", "name": "Name"}, inplace=True)
df.columns
```

```
Out[88]: Index(['Name', 'Age', 'designation', 'experience'], dtype='object')
```

